PA-consistency machines: comparison and a diagonal-vacuity calculus

Research report with reproducible verification artifacts

13 November 2026

Abstract

We compare the ProveIt U208 submissions, James Chang's historical PA209, and his current PA196 at fixed repository revisions. The nine U208 table presentations examined have two rooted labelled-graph isomorphism classes and share the same conditional-history reduction; that reduction cannot simply be subtracted from PA196 because its two relevant native states are already absent. We give a new diagonal-vacuity calculus, replacing one gate component by $B \to \forall x_B B$, and a written PA-internal calibration argument through the original proof compiler's interfaces. Its axiom program uses 191 rather than 201 macros and 3,700 rather than 3,812 primitive instructions. A complete 199-state binary table, exact startup, and all 4,815 symbolic primitive cases are supplied. **No machine with fewer than 196 states was obtained.** The existing 196-state construction remains the smallest validated table in this comparison. Finite computation, inherited kernel proofs, new kernel checks, and the additional written calibration arguments are explicitly distinguished.

Contents

1 Outcome and how to use the archive **3**

2 Frozen sources and comparison **3**
2.1 The 208-state submissions are not independent reductions 4
2.2 What changed from James's PA209 to PA196 4
2.3 Published formal statement versus the requested biconditional 4

3 A new diagonal-vacuity calculus **5**
3.1 What can be extracted from the new gate 5
3.2 Recovering reverse rewriting 5
3.3 Recovering the neighborhood-equality interface 6
3.4 PA calibration and logical scope 7
3.5 The exact emitter change 8

4 The reverse implication by proof translation **8**
4.1 Arithmetic interpretation 8
4.2 A uniform syntactic supplier for finite comprehension 9
4.3 The guarded gate has uniformly constructible PA proofs 9
4.4 Extracting the certificate from a halting run 10

5 Verification and certificate boundaries **10**
5.1 Literal tables and startup 10
5.2 A symbolic check for every primitive site 11
5.3 The inherited engine and a new kernel check 11
5.4 Source identity, folds, and rebuilding 12
5.5 What the evidence does and does not establish 12

6 Reduction searches and remaining work **13**
6.1 Why shortening the source did not reduce the state count 13
6.2 The local quotient family that was searched 13
6.3 Grammar search qualifications 14
6.4 Concrete directions for a subsequent attempt 14

A The complete 199-state transition table **15**

1 Outcome and how to use the archive

The numerical target was fewer than 196 working states, hence at most 195. It was not reached. In particular, the new 199-state table in this archive must not be presented as a smaller replacement for PA196.

The useful deliverables are: an exact comparison of the supplied designs; a new proof-calculus simplification with a complete written calibration argument; an explicit 199-state implementation of that calculus; an independent symbolic verifier; a kernel-checked weakening of the engine-agreement hypotheses; and reproducible bounded reduction searches. The unsuccessful searches establish no global lower bound.

Run `python3 verification/check.py -experiments` to reproduce the table, startup, and symbolic primitive checks; `python3 verification/check_emitter.py` checks the new axiom emitter; and `python3 verification/compare_proveit.py` reproduces the U208 graph census. The Python checks use only the standard library. A separate Lean 4.35.0 script checks the new generic engine lemma. The README describes file roles and the exact scope of each certificate.

2 Frozen sources and comparison

The task concerns the standard binary, one-tape model. The tape extends in both directions; every cell is initially zero; each ordinary transition writes one bit and moves exactly one cell left or right. A working state is a state with transitions. The separate halt label is excluded from the working-state count. No larger alphabet, nonblank input, or change of counting convention is used in this report.

The comparison freezes the following repository revisions, rather than the moving `main` branches:

- James Chang's current construction: `jamesyc/BusyBeaver` at `407cdc356f6b586635e71ac92be394255f4bfea4`.
- Its historical predecessor: the same repository at `69d98241a721ead20b4f0b17df9062950b40fbfe`.
- The ProveIt incoming reports and ConciseIPRA proposals: `VladimirReshetnikov/ProveIt` at `239b7a406e24405d464edc526e09d402abce5bc5`.

The source papers, theorem lists, transition files, and relevant implementation files were read directly from these snapshots. References are given at the end; local copies of the small inputs used by the reproducible checks accompany the report.

Construction	Working states	Rows	Principal change
U208 submissions' U209 parent	209	418	Earlier program layout
ProveIt U208 presentations	208	416	Conditional history sharing
James historical PA209	209	418	Published PA verification
James current PA196	196	392	Smaller calculus and new layout

Table 1: Working states exclude the halt label. Identical state counts do not imply identical transition graphs.

The label "U209 parent" here refers specifically to the parent supplied with the U208 submissions. We did not independently identify the exact earlier common transition table of that branch and James's historical PA209. The historical paper describes an earlier 211-state PA construction, identifies K230 as the source of its register engine, and cites PA238 for proof-calculus ingredients [2]. Shared construction ingredients alone do not identify an identical common whole-machine ancestor.

2.1 The 208-state submissions are not independent reductions

Eleven PA-related incoming ZIP archives were inspected. Among the U208 construction submissions, nine literal table presentations reduce to two rooted, symbol-labelled transition-graph isomorphism classes. This is an exact graph comparison: the checker starts at the declared initial state, follows the read-0 and read-1 successors in a fixed order, and retains both the written symbol and the direction. It does not erase transition labels or compare only sampled trajectories.

The two classes use the same underlying conditional-history idea. One presentation retains native state 172 and eliminates 211; the other retains 211 and eliminates 172. The latter redirects the old entry from native row $(116, 0)$ to 211 and replaces row $(211, 0)$ by a leftward visit to helper 217. Previously written bits distinguish the old roles. In the checked local simulations, the distinguishing histories include the two-bit history 10, and the exceptional macro can revisit the merged state itself. A search which requires helpers to avoid the merged state therefore misses this established reduction.

The included comparison receipt provides the original file hashes, the canonical hashes, and the complete renaming maps. The local-history checks were reproduced for both classes. These computations verify the stated finite reduction; they do not manufacture a new formal PA calibration for the older incoming reports.

2.2 What changed from James's PA209 to PA196

Let $p(a, b) = \binom{a+b+1}{2} + b$. The new encoding keeps

$$I(A, B) = 4p(A, B), \qquad M(a, b) = 4p(a, b) + 1, \qquad E(a, b) = 4p(a, b) + 3,$$

but changes the universal constructor to

$$U(a, B) = 4p(B, a) + 2.$$

The old constructor was $4p(a, B) + 2$. This is a deliberate transposition of the quantifier arguments, and it changes the arithmetic program used to assemble axiom instances.

PA196 also removes the old seventh equality head and adds guarded equality vacuity to the separation gate. Its remaining heads are reordered and, where helpful, propositionally rearranged. The number of items pushed by the axiom command, including the saved top, falls from ten to nine. The physical register program falls from 3,948 to 3,812 source sites. The historical command-level presentation collapses one physical site and consequently sometimes reports 3,947; that is a different count, not a discrepancy in the literal table.

The 196 states divide into 36 tape-engine states, 13 leaf-routing states, and 147 node-reader states. There are 159 distinct binary source nodes. The axiom body has 201 macro items, 68 distinct internal nodes, and six context-dependent pairs implemented with 62 reader states. Two obsolete helper states outside that body were also removed. The startup reaches its clean candidate-2 frame after 58,154 transitions, compared with 58,210 for the historical 209-state table.

The native states 172 and 211 used by the U208 reduction are both absent from PA196. Consequently, subtracting the U208 saving of one from the PA196 total is unjustified. The reductions overlap in the portion of the program that has already been replaced.

2.3 Published formal statement versus the requested biconditional

Write H_{196} for the literal arithmetic halting sentence and $N_{196} = \neg H_{196}$. The retrieved theorem list and README explicitly state the internal implication

$$\mathrm{PA} \vdash N_{196} \longrightarrow \mathrm{Con}(\mathrm{PA}).$$

The repository also proves standard-model nonhalting in Lean's metatheory and the relevant PA unprovability results. Its file `PA196/Logic/Soundness.lean` explicitly describes its semantic soundness argument as external. Those statements should not be silently relabelled as an already kernel-checked PA biconditional.

Section 4 supplies the syntactic argument needed for the reverse implication. That additional argument is a written proof construction; it is distinguished from the upstream checked theorem and from the finite machine checks run for this report.

3 A new diagonal-vacuity calculus

There is a mathematical simplification of the axiom program, even though the state-count target was not reached. Replace the equality-vacuity component of the gate by the diagonal clause

$$\Gamma(B) = I(B, U(B, B)), \qquad \text{that is,} \qquad B \longrightarrow \forall x_B \, B. \tag{1}$$

Here B is both a formula code and a variable index. This clause is sound because every variable index in the decoded formula B is strictly below B. Code zero is truth and presents no exceptional free variable.

For the implementation included in this archive, replace the old gate

$$F(a, b, c) = \neg(\Delta_c(a, b) \to (V(a, b, c) \to Q(a, b, c)))$$

by

$$F_\Gamma(a, b, c) = \neg(\Gamma(c) \to (V(a, b, c) \to Q(a, b, c))). \tag{2}$$

Keep the old guard $c \neq a, b$, the six other heads, the restricted propositional head, generalization, and modus ponens. Thus the membership-vacuity part of the gate is retained. Removing that part as well would invalidate the reverse-rewriting argument below.

3.1 What can be extracted from the new gate

Classical implicational reasoning gives both

$$F_\Gamma(a, b, c) \to \Gamma(c), \qquad F_\Gamma(a, b, c) \to G(a, b, c), \quad G = \neg(V \to Q).$$

Every diagonal instance $\Gamma(B)$ is derivable: take $c = B$ and $a = b = B + 1$. Both guard comparisons succeed. The extracted old gate G continues to give unrestricted membership vacuity at labels different from the atom's indices, and the original pointed-separation schema. Coincident labels $a = b$ are permitted by the guard and have already been accounted for in the arithmetic interpretation.

The retained propositional and quantifier heads give the same basic implicational reasoning, universal monotonicity, and the existential-elimination rule for a conclusion known to be vacuous at the quantified label. Membership vacuity extends to the quantifier-free membership matrices used below by the original negation and implication arguments. It also extends through a universal quantifier whose binder differs from the new vacuity label, by quantifier commutation. No equality-vacuity premise is used in those extensions.

3.2 Recovering reverse rewriting

The old proof used equality vacuity to discharge an existential witness from a conclusion containing an equality atom. The replacement is to make the separator property the negation of that entire conclusion.

Let $a \neq b$. Let $\chi(t)$ be one of $M(d,t)$, $M(t,d)$, or $M(t,t)$, with $d \neq a$ in the first two cases. The direct membership-rewriting heads give

$$E(v,y) \to \big(\chi(v) \to \chi(y)\big).$$

Put

$$C = E(a,b) \to \big(\chi(b) \to \chi(a)\big), \qquad \varphi = \neg C, \qquad c = \ulcorner \varphi \urcorner.$$

The code c exceeds all variable labels in C, so $c \neq a, b$; in the first two families it also gives $c \neq d$. There is no property-code collision requiring a separate padding branch.

Use pointed separation at (a, a, c). It gives

$$\exists x_c \, (S \wedge P), \qquad P = \varphi \to M(c,a), \qquad S \to U\big(a, M(c,a) \to \varphi\big). \tag{3}$$

Propositionally, φ implies $E(a,b)$, $\chi(b)$, and $\neg\chi(a)$. In particular, weakening the last consequent in (3) gives

$$S \to U\big(a, M(c,a) \to \neg\chi(a)\big).$$

The original negative-property specialization lemma now applies. It uses the equality witness head, the direct membership rewrites, and membership-only vacuity, and yields

$$S \to \big(M(c,b) \to \neg\chi(b)\big).$$

The equality atom occurring inside φ is never substituted: it was removed from the quantified consequent by the propositional weakening step.

Under φ, the point condition P gives $M(c,a)$. Direct rewriting under $E(a,b)$ gives $M(c,b)$, and the displayed consequence of S contradicts $\chi(b)$. Therefore

$$\varphi \to (S \to \neg P)$$

is derivable. Generalize at c, use universal distribution, and apply the available diagonal axiom $\Gamma(\varphi) = \varphi \to U(c,\varphi)$. This gives

$$\varphi \to U(c, S \to \neg P) = \varphi \to Q.$$

Pointed separation gives $\neg Q$. Hence $\neg\varphi$, and classical double-negation elimination gives C. This proof avoids a circular appeal to equality vacuity for C.

For $a = b$, the desired reverse rewrite is propositional. The cases $d = a$ reduce, as in the original proof, to direct rewrites and the already established diagonal case $\chi(t) = M(t,t)$. Thus for all labels the two unrestricted membership rewrites in both directions are recovered:

$$\begin{aligned} &E(a,b) \to \big(M(d,a) \leftrightarrow M(d,b)\big), \\ &E(a,b) \to \big(M(a,d) \leftrightarrow M(b,d)\big). \end{aligned} \tag{4}$$

3.3 Recovering the neighborhood-equality interface

Define

$$R(a,b,z) = U\big(z, M(a,z) \leftrightarrow M(b,z)\big) \wedge U\big(z, M(z,a) \leftrightarrow M(z,b)\big).$$

Let $z_0 = \ulcorner E(a,b) \urcorner$. Then $z_0 > a, b$, and the diagonal clause for $E(a,b)$ gives

$$E(a,b) \to U(z_0, E(a,b)).$$

Lifting (4) at z_0 gives $E(a,b) \to R(a,b,z_0)$.

It remains to recover the original interface at *every* $z \neq a, b$, not just at a numerically large binder. If $z = z_0$, nothing further is required. Otherwise consider either quantifier-free membership matrix

$$P(t) = M(a,t) \leftrightarrow M(b,t) \quad \text{or} \quad P(t) = M(t,a) \leftrightarrow M(t,b).$$

The two-direction rewrites imply $E(z,z_0) \to (P(z_0) \to P(z))$. The witness $\exists x_{z_0} E(z,z_0)$, universal distribution, and membership-only vacuity give

$$U(z_0, P(z_0)) \to P(z).$$

The antecedent is itself membership-only and vacuous at z, so generalization and distribution yield

$$U(z_0, P(z_0)) \to U(z, P(z)).$$

Apply this to both matrices. Consequently

$$E(a,b) \to R(a,b,z) \qquad (z \neq a, b)$$

is derivable without unrestricted vacuity for primitive equality.

The equality-witness head is unchanged. Existential monotonicity and the recovered neighborhood implication supply its interpreted witness form. These are exactly the original completeness proof's three interfaces: two-direction membership rewriting, neighborhood equality, and an equality witness, conventionally denoted (R), (N), and (W).

3.4 PA calibration and logical scope

The original PA-refutation compiler uses primitive equality through those interfaces, together with the old gate and the unchanged heads. In the inspected logical and arithmetic-certificate source closure, the uses of equality vacuity occur in the reverse-rewriting and neighborhood modules. Substitute the finite derivations above for those affected templates. Code formation, choosing $c = \ulcorner \neg C \urcorner$, and $z_0 = \ulcorner E(a,b) \urcorner$, the label-equality case splits, and the finite proof concatenations are primitive recursive. Their correctness is provable in PA. This gives the internal refutation-to-halting direction for the revised calculus.

In the other direction, each new diagonal clause has a uniformly generated first-order proof under the arithmetic interpretation, by the code-freshness lemma. Pointed separation is still supplied by full PA induction as in Section 4. The annotated proof-store argument and positive-length tape simulation then give the internal halting-to-refutation direction.

Thus the written calibration argument establishes the same target:

$$\mathrm{PA} \vdash (\neg H_\Gamma \leftrightarrow \mathrm{Con(PA)})$$

for a correct literal implementation of the revised program. This is a calibration to PA; merely replacing the searched system by iPRA would not supply it. The construction applies the general strategy of replacing a primitive schema by smaller derivably equivalent interfaces. Its decisive freshness lemma comes from PA196's existing numeric coding: the magnitude of a formula code guarantees a fresh variable label. We do not attribute that specific diagonal trick to the ConciseIPRA proposals.

3.5 The exact emitter change

Using the published macro names, replace the old half-open token interval $[121, 138)$ by

`copyA, copyA, copyA, cons, incT, incT, cons.`

The three copies push the old top and two copies of A. The first `cons` and two increments form $U(A, A) = 4p(A, A) + 2$; the last `cons` forms $I(A, U(A, A))$. The rest of the gate, its two numerical guard comparisons, and the final register cleanup are unchanged.

The independent free-algebra checker compares constructor trees, rather than finitely many numerical examples. It verifies that all nine stack entries are exactly the old entries and that the new top is precisely (2). The macro word has 191 items instead of 201; the primitive program has 3,700 sites instead of 3,812. The nine-item command extraction identities remain unchanged.

The supplied literal implementation has **199 working states**, after four validated context folds from a 203-state initial compilation. It has 398 transition rows, start state zero, halt label 199, and an all-zero initial tape. Its dense table has SHA-256

`6c29db7f324a0d18ee7dc5bfaa3f9b734011c453586d0bc1b96754a6bab4e52c.`

This is three states larger than the published PA196 machine and four states above the requested maximum of 195. The smaller logical program is therefore a genuine mathematical simplification, but *not* a solution of the requested state-count problem.

4 The reverse implication by proof translation

The desired reverse direction is $\mathrm{PA} \vdash H_{196} \to \neg\,\mathrm{Con(PA)}$. A semantic assertion that every PA theorem is true would be inappropriate here. Instead, translate finite certificates into finite PA proofs.

4.1 Arithmetic interpretation

Work temporarily in the conservative definitional extension of PA with exponentiation by two, whose totality PA proves. Let

$$\mathrm{Bit}(s, z) \;:\Longleftrightarrow\; \exists u, v\,[s = (2u + 1)2^z + v \;\wedge\; v < 2^z].$$

PA proves the elementary binary-arithmetic facts needed below, including $\mathrm{Bit}(s, z) \to z < s$, hence $\neg\,\mathrm{Bit}(x, x)$. The translation T of source formula codes is

$$\begin{aligned} T(0) &= (0 = 0), & T(M(a, b)) &= \mathrm{Bit}(x_a, x_b), \\ T(E(a, b)) &= (x_a = x_b), & T(U(a, B)) &= \forall x_a\, T(B). \end{aligned}$$

The first argument of M is the *container*; reversing this orientation would invalidate the separation argument. For implication, the decoder supplies $T(I(A, B)) \leftrightarrow (T(A) \to T(B))$. The special identity $I(0, 0) = 0$ is handled by the fixed proof of $(0 = 0) \leftrightarrow ((0 = 0) \to (0 = 0))$.

All recursive subcodes of a nonzero code are smaller. Every source variable label in a formula with code c is below c. Thus x_c is fresh in the decoded source formula. Bound variables introduced by the arithmetic translation are renamed by a primitive-recursive capture-avoiding operation; they do not invalidate this source freshness fact.

4.2 A uniform syntactic supplier for finite comprehension

For each arithmetic formula $\varphi(z, \vec{u})$, full PA induction proves the schema

$$\forall n\, \exists s < 2^n\, \forall z < n\, \big(\mathrm{Bit}(s, z) \leftrightarrow \varphi(z, \vec{u})\big). \tag{5}$$

For $n = 0$, use $s = 0$. Given a witness below 2^n, excluded middle for $\varphi(n, \vec{u})$ chooses $s + 2^n$ or s, respectively. Binary arithmetic verifies the old bits and the new bit. This uses a particular induction instance whose formula is constructed from the syntax of φ.

Crucially, there is a primitive-recursive procedure that inserts φ into this fixed proof template. Its correctness concerns proof codes. It neither evaluates φ nor asserts a uniform arithmetic truth predicate. Proofs in the conservative extension are subsequently translated back into ordinary PA by a fixed primitive-recursive proof transformation.

Apply (5) at $n = A + B + 1$ to

$$\psi(z) = (\mathrm{Bit}(A, z) \vee z = B) \wedge \varphi(z).$$

The resulting set code s satisfies pointed finite separation:

$$\begin{aligned}&\forall z\big[(\mathrm{Bit}(A, z) \to (\varphi(z) \to \mathrm{Bit}(s, z))) \wedge (\mathrm{Bit}(s, z) \to \varphi(z))\big],\\ &\varphi(B) \to \mathrm{Bit}(s, B).\end{aligned}$$

Elements of A are below A, and all bits of s above the chosen bound vanish. This explains why the bounded comprehension proof gives the displayed unbounded universal statement.

4.3 The guarded gate has uniformly constructible PA proofs

For source labels a, b and code c, put

$$\begin{aligned}&\Delta = E(a, b) \to U(c, E(a, b)), \qquad V = M(a, b) \to U(c, M(a, b)),\\ &P = c \to M(c, b),\\ &S = U\big(b, (M(a, b) \to P) \wedge (M(c, b) \to c)\big),\\ &Q = U(c, S \to \neg P), \qquad F = \neg(\Delta \to (V \to Q)).\end{aligned}$$

The axiom command emits F only when $c \neq a, b$; otherwise it emits code zero. The guard supplies the two vacuity proofs $T(\Delta)$ and $T(V)$. With $C = T(c)$, pointed separation for the capture-avoiding substitution $C[x_b := z]$ supplies $\exists x_c\, (T(S) \wedge T(P))$, hence $\neg T(Q)$, and therefore $T(F)$.

There is one label-collision case to check explicitly. If $a = b$, the first membership antecedent inside S is $\mathrm{Bit}(z, z)$, which PA proves false. The same comprehension witness still gives the second conjunct and the distinguished-point condition. Thus the argument covers coincident labels, as well as coincident values of distinct variables. The failed-guard branch supplies the fixed proof of $0 = 0$.

Source negation is implication to code one, whose translation is $\mathrm{Bit}(x_0, x_0)$. PA proves its negation uniformly, including under an enclosing binder with label zero. Fixed propositional templates therefore implement the classical negation, conjunction, and existential abbreviations used above.

All six remaining heads and the restricted propositional head have uniform proofs in first-order logic with equality: existential/universal elimination, quantifier commutation, the two membership rewrites, existence of an equality witness, universal distribution, and the classical implicational tautology. Modus ponens and generalization translate by the corresponding ordinary rules.

Consequently a primitive-recursive translator R can be specified with the syntactic correctness property

$$\mathrm{PA} \vdash \forall p \, (\mathrm{Prf}_{\mathrm{source}}(p, 1) \to \mathrm{Prf}_{\mathrm{PA}}(R(p), \ulcorner 0 = 1 \urcorner)). \tag{6}$$

The proof is induction on coded derivations and the decoding recursion, using the schema supplier above. This is a relation between proof codes, not a reflection principle.

4.4 Extracting the certificate from a halting run

Annotate the command store with proof codes for the two visible formula codes t, h and for every live stack entry. Initially all required formulas have code zero. The axiom command appends axiom proofs and a generalization of the previous h proof. A successful modus-ponens match appends the corresponding proof; its unsuccessful branch returns the code-zero proof. Shift moves proof references with the formulas; an empty-stack return again has code zero. The halt test is $t + h = 1$, so one of the two visible codes is one. This is why tracking only t would be insufficient.

These are primitive-recursive manipulations of annotated stores. The source macro contracts and scheduler transfer the invariant to the primitive register program. Substituting (6) for the external semantic non-refutability assertion yields the required PA refutation from source halting.

The final tape/source reflection uses positive-length simulation. The exact finite startup prefix first excludes a halt before the clean frame. Given a tape halting trace of length t, execute $t + 1$ source instructions from that frame. If no source halt has occurred, concatenating the positive-length instruction simulations gives a tape run longer than the alleged halting trace. Determinism and the absence of a transition at the halt label contradict this.

This argument can be formalized in PA: source stepping is primitive recursive, the unary register excursions have PA-provable terminating bounds, and finite trace concatenation is primitive recursive. Existing upstream files already supply coded source steps and forward tape lifts inside arbitrary PA models; retaining an accumulated length bound in that induction supplies the reflection just described.

Together with the published internal direction, this gives the written proof construction for

$$\mathrm{PA} \vdash (N_{196} \leftrightarrow \mathrm{Con}(\mathrm{PA})).$$

The new translator and the strengthened internal trace-lift theorem have not been implemented and built as additional Lean declarations in this report. The claim here is a mathematical proof construction, with its arithmetization steps made explicit, not a new claim about the scope of the upstream kernel certificate.

5 Verification and certificate boundaries

The statement that a machine is small, the statement that its tape program implements a source program, and the statement that the source calculus is calibrated to PA require different evidence. This archive keeps those obligations separate.

5.1 Literal tables and startup

The file `diagonal199/machine.tm` has exactly one row for every pair (q, b) with $0 \le q < 199$ and $b \in \{0, 1\}$. Its row format is

```
state  read  write  direction  next_state.
```

The alphabet is $\{0, 1\}$, the blank is zero, the initial state is zero, and the separate halt label is 199. Directions are `L` and `R`; there are no stationary moves. The companion native table retains the sparse internal labels, with initial state 74, decoder root 84, and halt label 254. The complete dense-to-native bijection is supplied and checked transition by transition. Counting the largest sparse label would be an incorrect state count.

The startup check runs the actual table on an all-zero tape. At transition 58,154 it checks the entire finite set of nonzero cells, head position, and native control state against the clean source frame. Its physical register vector is

$$(0, 0, 0, 0, 0, 0, 2, 0, 0) +\!+ (1, \ldots, 1),$$

where the final vector has 26 entries. The source address is the candidate-2 startup site and the address gap is 48. Checking only the control state would not establish this interface.

5.2 A symbolic check for every primitive site

The Python verifier is an independent port of the finite path and symbolic-execution definitions in the published `M6/Runtime.lean`. It represents unary runs by affine lengths and retains the arbitrary register file as an abstract segment. Consequently, an accepted primitive case describes all register values covered by its precondition; it is not a sample of small concrete register values.

For each source site the checker constructs the required input frame and output frame. Decrement instructions have separate zero and positive cases. The interpreter follows the literal table, checks exact tape effects, and requires positive concrete movement for the primitive execution. It may summarize register excursions through the inherited engine lemmas. This last operation makes the engine-agreement check essential: a modified engine row must never be hidden by an unchanged oracle.

Table	States	Source sites	Cases	Startup steps
Published PA196	196	3,812	4,959	58,154
Diagonal-vacuity machine	199	3,700	4,815	58,154
Negation peephole experiment	197	3,596	4,671	58,154
Original-calculus repacking	201	3,812	4,959	58,154

Table 2: Successful finite symbolic and blank-startup checks. The last two rows are unsuccessful state-reduction experiments, not new minima.

The maximum number of symbolic interpreter steps needed by a primitive case in these checks is 515. This number is not a uniform bound on the concrete running time: the summarized unary excursions can take arbitrarily many concrete steps as register values increase.

5.3 The inherited engine and a new kernel check

The original generic engine agreement asks for 44 transition rows, with one further append-one row supplied separately. Inspecting its proof dependencies showed that read-zero rows of native states 3, 16, and 40 need not be included in the generic agreement hypothesis. The archive's `M6/NativeMinimal.lean` proves the resulting engine theorem with 41 agreement rows, plus the separately checked row

$$(198, 0) \longmapsto (1, \mathrm{L}, 40).$$

The full generic proof was checked with Lean 4.35.0. Its reported axioms are `propext`, `Classical.choice`, and `Quot.sound`; no `sorryAx` occurs. The Python verifier explicitly checks all 42 fixed rows before using an engine summary.

An additional deletion audit tried omitting each of the remaining 41 agreement rows while leaving the proof otherwise unchanged. None of those 41 attempts compiled. That is a statement about this proof script, not a proof that every row is semantically indispensable. The newly freed rows did not yield a smaller table in the local searches.

For the upstream checking pipeline, the dependency closure of `PA196.CheckCoverage` was rebuilt and one 16-site coverage module was checked in the Lean kernel. The full 4,959-case baseline and 4,815-case new table were checked with the supplied Python interpreter. We did *not* generate and kernel-check a complete new family of Lean coverage modules for the 199-state machine.

5.4 Source identity, folds, and rebuilding

Checking primitive contracts alone would allow the wrong source program to be implemented perfectly. Therefore `verification/rebuild.py` also checks the following identities:

1. The axiom macro tree expands to the intended 191-item word, obtained by the exact replacement specified in Section 3.

2. Each inherited macro has its original ordinal control-flow graph. The outer program is unchanged except for relocating indices beyond the shortened axiom block. There are no hidden edges into a different interior instruction.

3. Compiling that tree with the frozen engine produces the initial 203-state table. The four listed quotients reproduce the final native 199-state table exactly.

4. Renumbering the final table by the supplied map reproduces every row of the dense binary table.

The four folds reduce the intermediate state counts to 202, 201, 200, and 199. Each was accepted only after all 4,815 symbolic cases and the exact startup passed. The corresponding pairs of native labels are

$$(5020, 5026), \quad (5021, 5025), \quad (5030, 5042), \quad (5057, 5058),$$

with the first label retained. The replacement rows are in `folds.json`. These pairs are meaningful only with the specific supplied grammar and its native numbering.

The free-algebra emitter check is complementary: it verifies the exact formulas constructed on top and in all nine stack slots, while the ordinal graph audit verifies the inherited macro and outer control flow. Neither check is a substitute for the calibration proof.

5.5 What the evidence does and does not establish

The finite checks establish the explicit table and its uniform primitive interfaces relative to the engine lemmas. The mathematical proof in Sections 3 and 4 supplies the PA calibration. The upstream repository supplies its own published formal theorems. The new generic engine theorem is kernel checked. The new calculus and the strengthened PA-internal converse remain written proof constructions, not newly completed Lean formalizations. These distinctions are material and should be retained when citing or extending this work.

6 Reduction searches and remaining work

6.1 Why shortening the source did not reduce the state count

The state cost is governed by shared binary subtrees and by compatible contexts in the tape decoder. It is not proportional to the number of source instructions. A new grammar can have fewer distinct nodes but lose several existing pairs of reader states that share a physical implementation. The decoder also imposes concrete path constraints: a superficially attractive 67-node balanced grammar had a zero run of length nine and failed the primitive checks. Valid layouts in this investigation retained maximum zero run six and the required bounded address depth.

A valid 67-node repacking of the original axiom word compiled to 201 states; the inherited 68-node layout contributes only 62 physical axiom readers after its six folds. Similarly, the new 191-item diagonal word admits useful logical simplification, but the best fully checked implementation obtained here still has 199 states. Raw grammar counts are therefore never reported as literal machine counts.

One safe arithmetic peephole replaces a `push; incT; cons` negation construction by an inherited drain-to-scratch block, an increment, and the inherited pairing/quadrupling block. It preserves the intended formula and shortens the original-calculus source to 3,596 sites. Its checked implementation has 197 states. Combining that peephole with the diagonal rule was explored at the grammar level, but produced no accepted smaller literal machine. Timeouts in those grammar searches are not infeasibility results.

6.2 The local quotient family that was searched

The local search retains a bounded tape window, records all symbolic primitive contexts and the concrete startup contexts, and tries replacing two reader states by one. The simulator follows the *modified quotient machine*, including revisits to the merged state. A trial succeeds only when its finite window, written bits, head displacement, and successor reproduce the original transition's local effect. Access to an unknown cell rejects that trial. Potential hits still require the independent full symbolic and startup checks.

For the principal search, the window has seven cells to the left of the head and three cells starting at the head. A detour has at most 71 transitions. The engine and designated interface states remain protected. Both replacement rows range over their write bits, directions, and permitted successors. This is a bounded local replacement family; it omits changes to larger portions of the machine, larger windows, longer detours, and different source semantics.

Search family	Parameter choices tested	Result
PA196, identical inherited rows	115,440	No hit
PA196, one inherited row	44,981,040	No hit
PA196, both rows synthesized	5,650,476,975	No hit
Diagonal199, both rows synthesized	6,176,511,792	No hit
Freed engine-entry rows, short window	536,640	No hit
Freed engine-entry rows, wider window	536,640	No hit

Table 3: Finite search results. Counts are loop parameter choices and need not be distinct quotient machines. The hypotheses of these families are not global restrictions on possible Turing machines.

Additional direct row merges, reader-to-engine merges, tree rotations, source abstract interpretation, and smaller detour families were explored. They produced no accepted table below 196. The

archive includes the main C++ search programs, their exact context inputs, and logs, so the largest bounded search can be repeated without treating the counts as a proof of minimality.

6.3 Grammar search qualifications

Integer-programming and local grammar searches were used to change association, head order, and shared subexpressions. For the diagonal word with all six old fold shapes and their incoming tags forced, one restricted model had minimum 69 raw axiom nodes. Other restricted targets were infeasible; broader targets ended without a feasible incumbent before their time limits. These outcomes are useful engineering evidence but do not establish a grammar-independent optimum. In particular, "no incumbent before timeout" must not be rewritten as "infeasible."

Source-level reachability pruning also did not solve the problem. In the final 199-state implementation every retained state appeared in a symbolic primitive execution or in the concrete startup trace. Those symbolic executions deliberately cover more states than a single actual blank-tape trajectory may visit. Observation in that family rules out the corresponding simple pruning argument; it does not prove that every state is visited in the actual infinite blank-tape computation.

6.4 Concrete directions for a subsequent attempt

The diagonal rule removes ten macro items without changing the nine-item stack protocol. A useful next optimization target is therefore a grammar for that exact word that recovers at least four further physical state savings, or a different calibration-preserving emitter with a better shared structure. The relevant objective is the number of physical readers after folds, subject to control-flow and primitive-contract checks.

A second route is to redesign the address decoder or register engine jointly with the grammar. The local searches hold those components mostly fixed. The three newly freed engine hypotheses may be useful in a more extensive redesign even though they did not yield a one-state merge here.

A third route is to use the concise-IPRA proposals as a source of coding and freshness ideas while separately proving the two refutation translations required for PA calibration. The diagonal clause is an example. Replacing the searched theory by iPRA without such a translation changes the consistency sentence and does not meet the task.

Finally, formalizing the two syntactic translators and the positive-length internal simulation would remove the principal gap between the written biconditional argument and a single complete kernel-checked theorem for a future smaller machine. That formalization would also make calculus changes safer to evaluate. None of these directions is a claim that a 195-state machine has already been constructed.

A The complete 199-state transition table

This is the dense table in `diagonal199/machine.tm`. A cell such as `1R74` means write one, move right, and enter state 74. Start in state 0 on the all-zero tape. Label 199 means halt and is not a working state. The nine columns below show three consecutive state ranges side by side; each range has its own read-zero and read-one columns.

State	Read 0	Read 1	State	Read 0	Read 1	State	Read 0	Read 1
0	1R1	1R61	67	0R53	1L72	134	0R135	1R139
1	1R82	1R1	68	0R73	1R74	135	0R12	1R136
2	0L2	1L3	69	0R69	1R75	136	0R86	1R137
3	0L4	1L3	70	0R76	1R77	137	0R138	1R58
4	0L6	1L3	71	0R79	1L110	138	0R78	1R58
5	1R7	0L5	72	0L98	1R54	139	0R140	1R198
6	0L9	1L3	73	0R48	0R33	140	0R141	1R189
7	0R7	0R10	74	0R81	0L41	141	0R64	1R142
8	0R14	1R13	75	0R50	1R132	142	0R143	1R187
9	0L15	1L3	76	0R130	1R21	143	0R144	1R178
10	0L16	0R10	77	0R101	1R90	144	0R12	1R145
11	0L26	1R18	78	0R92	1R93	145	0R146	1R177
12	0R19	1R20	79	0R73	1R72	146	0R86	1R147
13	0L79	1L31	80	0R91	1R119	147	0R148	1R172
14	0L11	0R22	81	0R95	1R96	148	0R149	1R164
15	0L23	1L3	82	0R62	0R1	149	0R78	1R150
16	1L2	0L2	83	0R27	1R97	150	0R151	1R163
17	0R24	0R32	84	0R89	1R88	151	0R126	1R152
18	0R84	0L95	85	1L99	1L85	152	0R153	1R154
19	0R28	1R29	86	0R19	1R100	153	0R154	1R156
20	0R30	1R31	87	0R58	1R58	154	0L145	1R155
21	0R24	0R57	88	0R117	1R115	155	0L148	1R78
22	0R33	1R22	89	0L26	1R134	156	0R157	1R158
23	0L34	1L3	90	0R95	0L131	157	0R116	1R155
24	0L35	0R1	91	0R102	1R103	158	0R155	1R135
25	0R36	1R25	92	0R67	1R56	159	0R160	1R161
26	0R122	1R41	93	0R55	1R104	160	0R161	1R136
27	0R38	0L39	94	0R21	1R105	161	0R19	1R162
28	0R40	1R41	95	0R21	0L10	162	0R58	1L195
29	0R42	1R43	96	0R59	1R41	163	0R164	1R166
30	0R44	1R45	97	0R106	1R8	164	0R12	1R165
31	0R21	1R46	98	0R91	1R107	165	0R116	1R137
32	0R47	1R32	99	0L131	0L85	166	0R167	1R168
33	0R32	1R33	100	0R108	1R94	167	0R136	1R135
34	0L69	1L3	101	0L109	0L2	168	0R169	1R170
35	1R35	1L2	102	0R38	1R68	169	0R165	1R78
36	0R49	1R36	103	0R59	1R110	170	0R12	1R171
37	0R28	1R20	104	0R59	1R111	171	0R86	1R138
38	0L11	0R36	105	0R66	0L16	172	0R173	1R176
39	0L5	1L51	106	0R14	1R110	173	0R162	1R175
40	0R24	0R25	107	0R21	1R112	174	0R116	1R171
41	0R13	0L16	108	0R113	1L20	175	0R158	1R58
42	0R53	1R54	109	0R30	1R65	176	0R157	1R173
43	0R55	1R56	110	0R38	1L108	177	0R159	1R174
44	0R24	0R52	111	0R121	0L16	178	0R179	1R168

State	Read 0	Read 1	State	Read 0	Read 1	State	Read 0	Read 1
45	0R58	1R59	112	0R114	0L16	179	0R180	1R186
46	0R60	0L16	113	0R24	0R47	180	0R181	1R182
47	0R25	1R47	114	0R48	0R32	181	0R58	1R157
48	0L35	0R61	115	0R70	1R86	182	0R167	1R183
49	0R16	1R49	116	0R19	1R118	183	0R155	1R184
50	0R63	1R64	117	0R126	1R28	184	0R116	1R185
51	0L39	1L51	118	0R122	1R107	185	0R116	1R169
52	0R22	1R52	119	0R94	1R86	186	0R170	1R170
53	0L11	0R49	120	0R83	1R98	187	0R188	1R188
54	0R58	0L16	121	0R87	1R87	188	0L156	1R174
55	0R65	1R66	122	0R17	1L109	189	0R190	1R196
56	0R59	1R67	123	0R70	1R101	190	0R116	1R191
57	0R52	1R57	124	0R96	1R108	191	0R192	1R194
58	0R48	0R25	125	1L39	1L99	192	0R193	1R78
59	0L28	1L68	126	0R80	1R78	193	0R78	1R161
60	0L102	1L30	127	0R120	1R133	194	0R160	1R195
61	0R0	1R61	128	0R124	1R123	195	0R188	1R192
62	1R125	1L51	129	0R123	1R128	196	0R169	1R197
63	0R70	1R199	130	0R44	1R90	197	0R193	1R122
64	0L126	1R31	131	0L5	0L131	198	0R129	0L131
65	0R40	1R60	132	0R127	1R59			
66	0L79	1R71	133	0R11	1R37			

References

[1] James Chang, *Nonhalting and PA-independence for a 196-state Turing machine*, repository and paper at commit `407cdc356f6b586635e71ac92be394255f4bfea4`.
`https://github.com/jamesyc/BusyBeaver/tree/407cdc356f6b586635e71ac92be394255f4bfea4`

[2] James Chang, historical 209-state construction, commit `69d98241a721ead20b4f0b17df9062950b40fbfe`.
`https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe`

[3] Vladimir Reshetnikov, ProveIt incoming research archives, frozen revision `239b7a406e24405d464edc526e09d402abce5bc5`.
`https://github.com/VladimirReshetnikov/ProveIt/tree/239b7a406e24405d464edc526e09d402abce5bc5/docs/incoming`

[4] Vladimir Reshetnikov, ProveIt ConciseIPRA proposals, same frozen revision.
`https://github.com/VladimirReshetnikov/ProveIt/tree/239b7a406e24405d464edc526e09d402abce5bc5/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`

[5] The current repository's theorem list and formal-source files, especially `docs/result.md`, `PA196/Logic/CommandSoundness.lean`, `PA196/PhysicalEndpoint.lean`, `PA196/Arithmetic/Machine/EndpointInternalCodedSourcePhysicalStep.lean`, and `M6/Runtime.lean`, at the revision in [1].